

INFO8006 Introduction to Artificial Intelligence

Exercise session 2: Games and adversarial search

RECAP

Games

A *game* is defined by

- the *states* $s \in \mathcal{S}$ and the *initial state* $s_1 \in \mathcal{S}$;
- the *actions* $\text{actions}(s) \subseteq \mathcal{A}$ available in state s ;
- the *transition model* $s' = \text{result}(s, a)$, the state reached by taking a in s ;
- a function $\text{player}(s) \in \{1, \dots, N\}$ that says whose turn it is in state s ;
- the *terminal states* $T \subseteq \mathcal{S}$, tested by $\text{terminal-test}(s)$, *i.e.* $s \in T$;
- a *utility* $\text{utility}(s, p)$, the payoff of player p when the game ends in $s \in T$.

The *game tree* is defined by s_1 , actions and result. A solution is a *strategy* $\pi_p : \mathcal{S} \rightarrow \mathcal{A}$, a move for every state the opponent can lead to.

Zero-sum games

A game is *zero-sum* if the utilities of the players sum to the same constant c in every terminal state, $\sum_{p=1}^N \text{utility}(s, p) = c$ for all $s \in T$. With two players, $\text{utility}(s, 2) = c - \text{utility}(s, 1)$: a single utility describes the game, which player 1 maximizes and player 2 minimizes. Below, the game is deterministic, turn-taking, two-player, zero-sum and of perfect information: MAX maximizes the utility, MIN minimizes it, and MAX moves first.

Minimax

The *minimax value* of s is the best payoff MAX can secure against an optimal MIN,

$$\text{minimax}(s) = \begin{cases} \text{utility}(s) & \text{if } s \in T \\ \max_a \text{minimax}(\text{result}(s, a)) & \text{if MAX plays} \\ \min_a \text{minimax}(\text{result}(s, a)) & \text{if MIN plays,} \end{cases}$$

over $a \in \text{actions}(s)$, and the optimal move is $\pi^*(s) = \arg \max_a \text{minimax}(\text{result}(s, a))$. Time $O(b^m)$, space $O(bm)$.

α - β pruning

α and β are the best values found so far for MAX and MIN on the path from the root: a MIN node stops as soon as its value $v \leq \alpha$, a MAX node as soon as $v \geq \beta$. The minimax values are unchanged and, with a perfect move ordering, $O(b^{m/2})$ nodes are examined. A *transposition table* stores $\text{minimax}(s)$ for every state s already evaluated, so that a state reached again by another sequence of moves is looked up rather than searched again.

H-Minimax

Under time constraints, an *evaluation function* $\text{eval}(s)$ replaces the utility and a *cutoff test* at depth d replaces the terminal test,

$$\text{h-minimax}(s, d) = \begin{cases} \text{eval}(s) & \text{if cutoff} \\ \max_a \text{h-minimax}(\text{result}(s, a), d + 1) & \text{if MAX plays} \\ \min_a \text{h-minimax}(\text{result}(s, a), d + 1) & \text{if MIN plays.} \end{cases}$$

The evaluation should order states as their minimax values do, and the cutoff apply to *quiescent* states only; what lies beyond the depth limit is hidden (*horizon effect*). An inaccurate evaluation can be compensated by a deeper search, as long as it becomes more accurate closer to the end of the game, but not the converse.

Multi-player games

Without the zero-sum assumption, or with $N > 2$ players, minimax backs up a vector of utilities $v(s) \in \mathbb{R}^N$, where the player to move maximizes its own component,

$$v(s) = \begin{cases} (\text{utility}(s, 1), \dots, \text{utility}(s, N)) & \text{if } s \in T \\ v(\text{result}(s, a^*)) & \text{otherwise,} \end{cases} \quad a^* = \arg \max_a v_{\text{player}(s)}(\text{result}(s, a)).$$

Expectiminimax

A *chance node* stands for a state-action pair (s, a) , from which the next state s' follows with probability $P(s' \mid s, a)$. With $v(s) = \text{expectiminimax}(s)$,

$$v(s) = \begin{cases} \text{utility}(s) & \text{if } s \in T \\ \max_a q(s, a) & \text{if MAX plays} \\ \min_a q(s, a) & \text{if MIN plays,} \end{cases} \quad q(s, a) = \sum_{s'} P(s' \mid s, a) v(s'),$$

where $q(s, a)$ is the value of the chance node (s, a) , and $P(\text{result}(s, a) \mid s, a) = 1$ for a deterministic action. With chance nodes, an evaluation function must be a positive linear transformation of the expected utility: preserving the order of states is not enough.

Monte Carlo tree search

A *random playout* plays the game to its end with random moves; the proportion of playouts won evaluates a state. Each node n of the search tree keeps the wins $Q(n, p)$, over the playouts through n , of the player p who moved into n , and its visits $N(n)$. A round does *selection*, *expansion*, *simulation* and *backpropagation*; selection picks the child n' of n that maximizes

$$\frac{Q(n', p)}{N(n')} + c \sqrt{\frac{\log N(n)}{N(n')}},$$

where p is the player to move in n . The move played is the one leading to the most visited child of the root.

In session: exercises 1, 2, 4 (b)–(d).

Exercise 1 Tic-tac-toe (AIMA ch. 5)

You are writing the engine of a program that plays Tic-Tac-Toe against human players. Tic-Tac-Toe is a game for two players, X and O, who take turns marking the cells of a 3×3 grid, X moving first. The player who succeeds in placing three of their marks in a straight line (horizontal, vertical or diagonal) wins the game. If neither of the players wins before the grid is full, it is a draw.

The program plays X, the MAX player, and its opponent O, the MIN player. We define X_n as the number of rows, columns or diagonals with exactly n X's and no O's. Similarly, O_n is the number of rows, columns, or diagonals with just n O's. A position s is terminal if $X_3(s) \geq 1$, $O_3(s) \geq 1$ or if the grid is full. The utility function assigns +1, -1 or 0 to such position, respectively. For non-terminal positions, we use an evaluation function defined as $\text{eval}(s) = 3X_2(s) + X_1(s) - 3O_2(s) - O_1(s)$.

- (a) Define the search problem associated with the Tic-Tac-Toe game.

Solution. Let a state of the game be a matrix $s = (s_{ij}) \in \{-1, 0, 1\}^{3 \times 3}$, where the values -1, 0 and 1 respectively denote O, empty and X cells. X is MAX and O is MIN.

Initial state

$$s_1 = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}$$

Player function X moves when there are as many X's as O's on the grid, and O otherwise,

$$\text{player}(s) = \begin{cases} \text{MAX} & \text{if } \sum_{i,j} s_{ij} = 0 \\ \text{MIN} & \text{otherwise.} \end{cases}$$

Actions An action is represented by the position (i, j) of an empty cell in the grid. Then, the set of possible actions is $\text{actions}(s) = \{(i, j) : s_{ij} = 0\}$.

Transition model For an action $(i, j) \in \text{actions}(s)$, $\text{result}(s, (i, j)) = s'$ is the same as s except that $s'_{ij} = 1$ if $\text{player}(s) = \text{MAX}$ and -1 otherwise.

Terminal test $\text{terminal-test}(s)$ is true if $X_3(s) + O_3(s) \geq 1$ or $\prod_{i,j} s_{ij} \neq 0$, false otherwise.

Utility function $\text{utility}(s, \text{MAX}) = 1$ if $X_3(s) \geq 1$, -1 if $O_3(s) \geq 1$ and 0 otherwise.

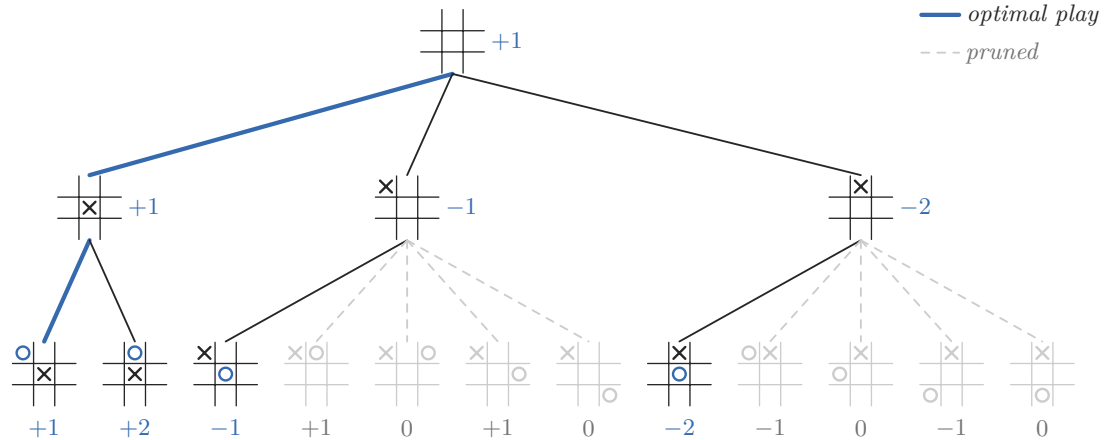
- (b) Approximately how many possible game states of Tic-Tac-Toe are there?

Solution. Every cell being empty, X or O, there are at most $3^{3 \times 3} = 19\,683$ states, unreachable ones included, and at most $9! = 362\,880$ games, as many games end before the grid is full. The exact counts are 5478 reachable states and 255 168 games.

- (c) Show the whole game tree starting from an empty grid down to depth 2 (one X and one O on the board), taking symmetry into account.
- (d) Annotate your tree with the evaluations of all the positions at depth 2.

- (e) Using the H-Minimax algorithm, annotate your tree with the backed-up values for the positions at depths 1 and 0, and use those values to choose the first move of the program.

Solution. The tree below answers (c) to (e). The center is worth +1, a corner -1 and an edge -2, so X starts in the center.

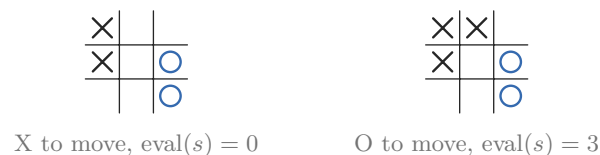


- (f) Mark the nodes that α - β pruning would not evaluate if the nodes were generated in the optimal order.

Solution. The faded nodes in the tree of (e). With the center first, $\alpha = +1$ at the root once its two children are evaluated. The corner and the edge then start with their worst child for X, of values -1 and -2, both $\leq \alpha$, and their four other children are pruned.

- (g) Is this evaluation function a good heuristic? If not, provide one or more states s for which $\text{eval}(s)$ is misleading.

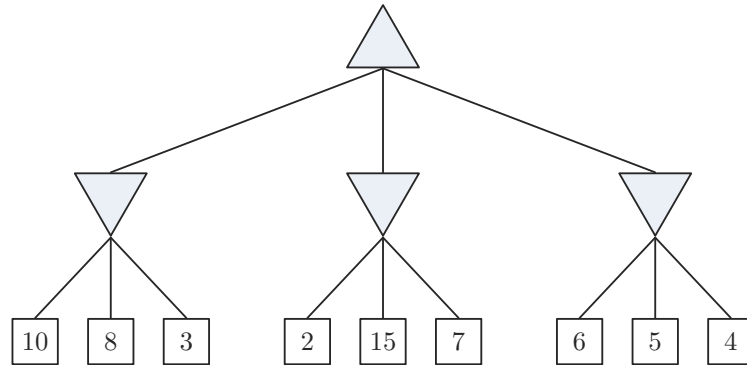
Solution. This is not a good heuristic, mainly because it does not take into account which player's turn it is. This results in states for which X is winning with lower evaluation than other states for which O is winning. In the states below, X is to move on the left and wins at once, yet $\text{eval}(s) = 0$; O is to move on the right and wins at once, yet $\text{eval}(s) = 3$.



This heuristic does not preserve the Minimax *ordering* of intermediate states.

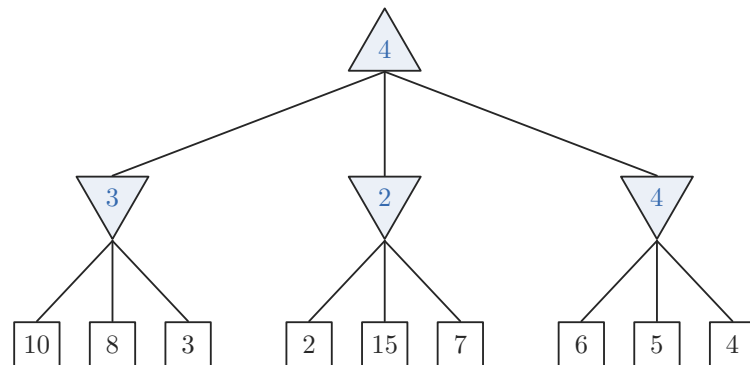
Exercise 2 Defending a network (CS188, Fall 2019)

A security program protects a computer network. It first chooses one of three configurations of the network; an attacker program, which observes the configuration, then chooses one of three attacks. The leaves of the tree below score the damage prevented: the defender maximizes it and the attacker minimizes it, so the game is zero-sum. The triangle that points up, at the root, is the choice of the defender (MAX); the triangles that point down are the choices of the attacker (MIN).



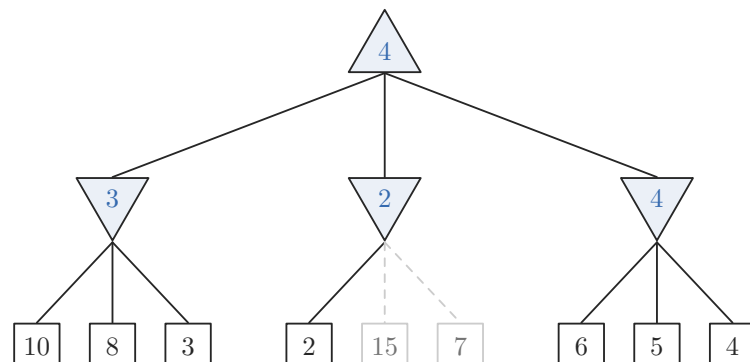
- (a) Assuming both programs act optimally, fill in the minimax value of each node. Which configuration does the defender choose?

Solution. The defender chooses the third configuration, which guarantees a score of 4 whatever the attack.



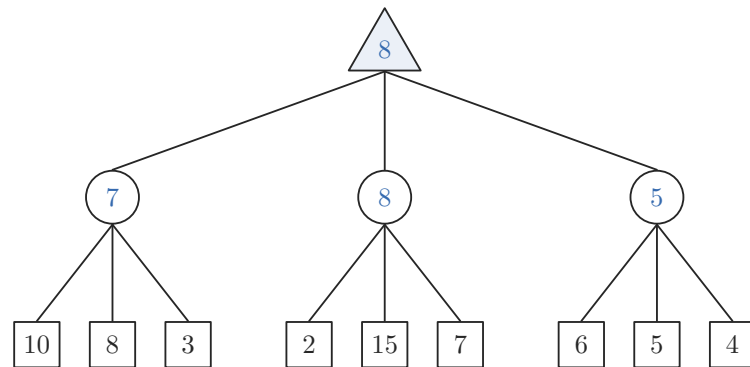
- (b) Which nodes can be pruned from the game tree above through α - β pruning? If no nodes can be pruned, explain why not. Assume the search goes from left to right; when choosing which child to visit first, choose the left-most unvisited child.

Solution. The first MIN node is worth 3, so $\alpha = 3$ at the root. The first leaf of the second MIN node is $2 \leq \alpha$: MAX will not play this node whatever its other leaves, and 15 and 7 are pruned. The leaves of the third MIN node, 6, 5 and 4, all stay above $\alpha = 3$, so all three are evaluated.



- (c) Suppose now that there is no attacker, but random failures: once the configuration is chosen, one of its three outcomes happens uniformly at random. The MIN nodes become chance nodes. Fill in the expectiminimax value of each node. Which configuration does the defender choose, and why does it differ from (a)?

Solution. The defender now chooses the second configuration, of expected score 8, although its worst outcome scores 2. Against random failures, the average outcome matters; against an attacker, who always picks the worst outcome for the defender, only the worst case does.

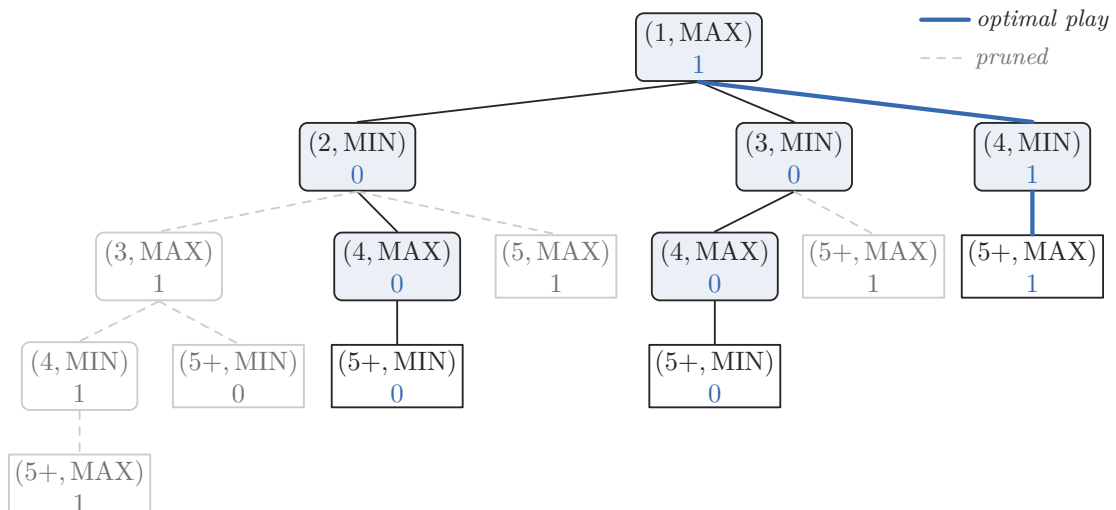


Exercise 3 21 misère game (Exam, January 2019)

A game is *solved* when the outcome of perfect play from its initial state is known, and a program that plays perfectly never loses; checkers was solved in 2007. We want to solve a small game. The game “21” is played with any number of players who take turns increasing a counter. The counter starts at 1 and each player in turn increases the counter by 1, 2, or 3; the player who says “21” or larger loses. Such a game, where the player who makes the last move loses, is a *misère* game. With two players, a state is a pair (v, p) of the value v of the counter and the player $p \in \{\text{MAX}, \text{MIN}\}$ to play, MAX playing first.

- (a) Consider the 2-player game of “5”, which has the same rules as “21” except that the player who says 5 or larger loses. Show the whole game tree.
- (b) Using the Minimax algorithm, annotate your tree with the backed-up values, and use those values to choose the optimal starting move. Who wins the game of “5” with perfect play?

Solution. The tree below answers (a) and (b). The utility of MAX is 1 when MIN says 5 or larger, *i.e.* in the terminal states (v, MAX) , and 0 otherwise. A node $(5+, p)$ merges the terminal states that the moves of its parent reach with a counter of 5 or larger, which all have the same value. MAX starts by adding 3, and the root is worth 1: the first player wins with perfect play, and the game of “5” is solved.

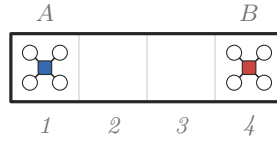


- (c) Mark the nodes that α - β pruning would not evaluate if the nodes were generated in the optimal order.

Solution. The faded nodes in the tree of (b). With (4, MIN) first, $\alpha = 1$ at the root. The nodes (2, MIN) and (3, MIN) then start with their child (4, MAX), of value $0 \leq \alpha$, and their other children are pruned.

Exercise 4 Drone race (AIMA ch. 5)

Two delivery drones, A and B , race in a narrow tunnel of four cells, shown below: A starts in cell 1 and must reach cell 4, and B starts in cell 4 and must reach cell 1. The drones move in turn, A first. At each turn, a drone must move to an adjacent free cell, in either direction. If the other drone occupies an adjacent cell, a drone may fly over it to the next free cell, if any. For example, if A is on 3 and B is on 2, then A may fly back to 1. The race ends when a drone reaches the opposite end of the tunnel. If A reaches cell 4 first, the value of the race to A is $+1$; if B reaches cell 1 first, it is -1 .



- (a) Define the search problem associated with this race.

Solution. Let a state of the game be a tuple $s = (i, j, p) \in \{1, 2, 3, 4\}^2 \times \{A, B\}$, where i and j are the respective positions of A and B and p is the player whose turn it is.

Initial state $s_1 = (1, 4, A)$.

Player function $\text{player}(s = (i, j, p)) = p$.

Actions $\text{actions}(s) \subseteq \{\text{left}, \text{right}\}$. For the player to move at x , with its opponent at y , “right” is available if $\text{next}(x, y) \leq 4$ and “left” if $\text{prev}(x, y) \geq 1$, where next and prev , defined below, give the cell reached, jump included.

Transition model For an available action $a \in \text{actions}(s)$,

$$\text{result}(s = (i, j, p), a) = \begin{cases} (\text{next}(i, j), j, B) & \text{if } p = A \text{ and } a = \text{right} \\ (\text{prev}(i, j), j, B) & \text{if } p = A \text{ and } a = \text{left} \\ (i, \text{next}(j, i), A) & \text{if } p = B \text{ and } a = \text{right} \\ (i, \text{prev}(j, i), A) & \text{if } p = B \text{ and } a = \text{left} \end{cases}$$

where

$$\begin{aligned} \text{next}(x, y) &= \begin{cases} y + 1 & \text{if } x + 1 = y \\ x + 1 & \text{otherwise} \end{cases} \\ \text{prev}(x, y) &= \begin{cases} y - 1 & \text{if } x - 1 = y \\ x - 1 & \text{otherwise} \end{cases} \end{aligned}$$

Terminal test $\text{terminal-test}(s = (i, j, p))$ is true if $i = 4$ or $j = 1$, false otherwise.

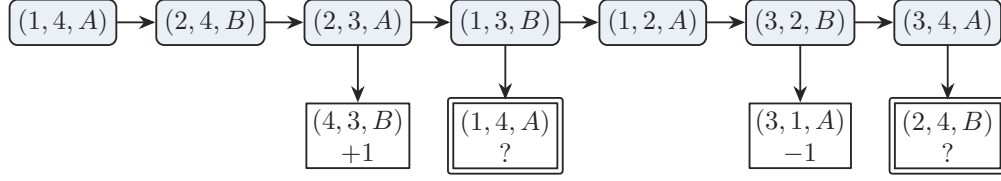
Utility function

$$\text{utility}(s = (i, j, p), A) = \begin{cases} +1 & \text{if } i = 4 \\ -1 & \text{if } j = 1 \end{cases}$$

(b) Draw the complete game tree, using the following conventions:

- Put each terminal state in a square box and annotate it with its game value.
- Put loop states (states that already appear on the path to the root) in double square boxes. Since their value is unclear, annotate them with a “?” symbol.

Solution.

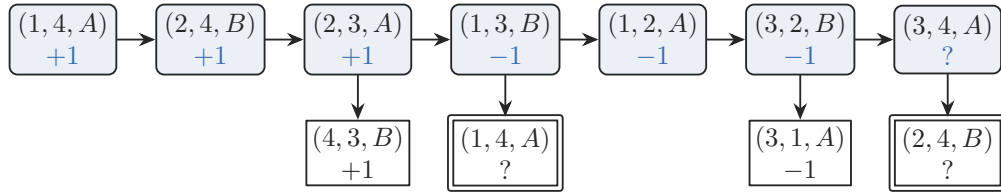


(c) Explain why the standard minimax algorithm would fail on this game.

Solution. Minimax recurses depth-first down to the terminal states, which requires a finite game tree. Here, a state can appear again further down a path (the loop states of (b)), so the game tree is infinite and the recursion never terminates on a branch that loops.

(d) Annotate each node with its backed-up minimax value. Explain how you handled the “?” values and why.

Solution. We handle the “?” values by assuming that a player only chooses to go in loop states if no winning action is available. That is, A (resp. B) prefers $+1$ (resp. -1) to “?”, but “?” to -1 (resp. $+1$).



Our strategy is equivalent to replacing “?” by any value strictly between -1 and $+1$, which only works for this game tree. For other games, it is not clear how to compare “?” values with intermediate outcomes like draws, wins of different degrees (as in score games) or even other “?” values. In such cases, algorithms more complex than Minimax must be used.

(e) This race can be generalized to a tunnel of n cells for any $n > 2$. Prove that A wins if n is even and loses if n is odd.

Solution. Let P be A if n is even and B if n is odd, and Q the other player. We show that P wins by always moving towards its goal, the cell where Q starts. Let d be the number of free cells between the two drones, as long as they have not crossed.

- Q never jumps before the crossing. $d = n - 2$ at the first turn of A and $d = n - 3$ at the first turn of B , so d is even at the first turn of P . Each move of P decreases d by 1 and each move of Q changes it by ± 1 , so d is even at every turn of P and odd at every turn of Q . Q therefore never faces an adjacent drone, the only case where it could jump.
- P can always move forward. It steps into a free cell if $d > 0$ and jumps over Q if $d = 0$. The jump is impossible only if Q stands on the goal of P , at the end of the tunnel, but Q could only have moved there from the cell that P now occupies, or by jumping over P , and neither is possible.
- The crossing. Let P jump on its $(m + 1)$ -th move, after m moves of one cell. Meanwhile, Q made m moves (if $P = A$) or $m + 1$ (if $P = B$), each bringing it at most one cell closer. The drones started $n - 2$ free cells apart, so $n - 2 \leq 2m + 1$, i.e. $n - 3 - m \leq m$. After the jump, P is $n - 3 - m$ cells from its goal and Q , to move, is $m + 1$ cells from its own.

- *The race.* No drone stands between Q and its goal any more, so Q needs at least $m + 1$ moves, while P reaches its goal in $n - 3 - m \leq m$ moves, before the $(m + 1)$ -th move of Q . Q can get in front of P again only by jumping over it, two cells away from its own goal; P then jumps over Q in turn or, if Q landed on the goal of P , steps back and jumps over Q once Q , forced to move, has left that cell.

P thus wins: A wins if n is even and B if n is odd.

Exercise 5 Chess and transposition table (AIMA ch. 5)

Chess engines store the positions they have already evaluated in a transposition table, since many move orders lead to the same position. Suppose you write a chess engine that evaluates 16 million nodes per second (Deep Blue, which beat Garry Kasparov in 1997, evaluated about 200 million).

- (a) Decide on a compact representation of a game state for storage in a transposition table.

Solution. There are 32 pieces, each on one of the 64 squares of the board, which takes 6 bits ($2^6 = 64$); a captured piece can be put on the square of its own king, which is never captured. This makes $32 \times 6 = 192$ bits, *i.e.* 24 B per state. This is a lower bound: a complete entry also stores the side to move, the castling and en passant rights, the promoted pieces, which fixed piece slots cannot encode, and the value found by the search with its depth.

- (b) About how many entries can you fit in a 4 GB in-memory table?

Solution. We can store roughly $4 \times 10^9 \times \frac{1}{24} \approx 1.7 \times 10^8$ states in the table.

- (c) Will that be enough for the three minutes of search allocated for one move?

Solution. It is not enough to store all $16 \times 10^6 \times 3 \times 60 = 2.88 \times 10^9$ evaluated nodes, about 17 times more.

- (d) How many table lookups can you do in the time it would take to do one evaluation? Suppose that you have a 3.2 GHz machine and that it takes 20 operations to do one lookup on the transposition table.

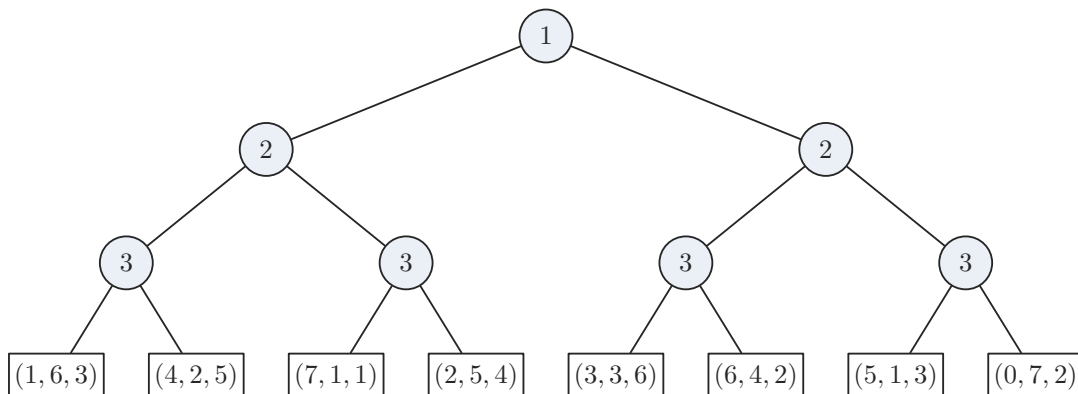
Solution. We can perform

$$\frac{3.2 \times 10^9}{16 \times 10^6} \times \frac{1}{20} = 10$$

table lookups in the same amount of time as a single evaluation. A lookup thus costs a tenth of an evaluation, and a hit saves at least that evaluation: the table pays off as soon as more than one lookup in ten finds its state.

Exercise 6 Self-driving cars at an intersection

Three self-driving cars reach an intersection and decide in turn how to cross it, each between two maneuvers and after observing the choices of the cars before it: car 1 at the root of the tree below, car 2 at the second level and car 3 at the third. Each leaf gives the time saved by the three cars, (u_1, u_2, u_3) , and each car maximizes its own.



- (a) Back up the utilities to the root. Which maneuver does car 1 choose? Is the game zero-sum?

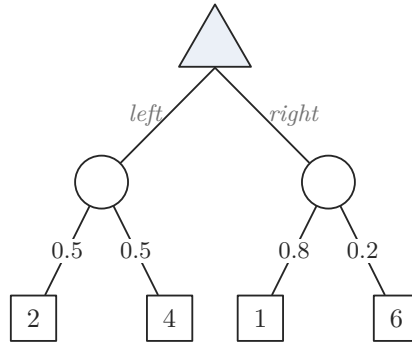
Solution. The nodes of car 3 are worth, from left to right, $(4, 2, 5)$, $(2, 5, 4)$, $(3, 3, 6)$ and $(5, 1, 3)$, those of car 2 are worth $(2, 5, 4)$ and $(3, 3, 6)$, and the root $(3, 3, 6)$: car 1 takes the right maneuver. The game is not zero-sum, as the total time saved varies from 9 to 12 across the leaves.

- (b) Car 1 now drives defensively: it assumes that the two other cars act against it and runs minimax on u_1 , as MAX, with cars 2 and 3 as MIN. Which maneuver does it choose, and what does it get if cars 2 and 3 in fact drive as in (a)?

Solution. The nodes of car 2 are worth $\min(1, 4, 7, 2) = 1$ on the left and $\min(3, 6, 5, 0) = 0$ on the right, so car 1 takes the left maneuver. Cars 2 and 3 then lead to $(2, 5, 4)$, and car 1 saves 2 instead of 3. Minimax guarantees at least 1, but its assumption is wrong here: cars 2 and 3 maximize their own time saved, not the loss of car 1.

Exercise 7 Evaluation functions in backgammon

A backgammon program chooses a move, after which the dice are rolled. Its evaluation function, learned from games against itself as in TD-Gammon, scores the positions reached, on the leaves of the tree below; the probabilities of the dice outcomes are on the edges.



- (a) Compute the expectiminimax value of each node. Which move does the program choose?

Solution. The chance nodes are worth $0.5 \times 2 + 0.5 \times 4 = 3$ and $0.8 \times 1 + 0.2 \times 6 = 2$, so the program moves left, for a value of 3.

- (b) The developers rescale the scores x of the evaluation function, into $f(x) = 2x + 1$ or into $g(x) = 2^x$, which both keep the ranking of the positions. Which move does the program choose in each case?

Solution. With f , the leaves become 5, 9, 3 and 13, and the chance nodes are worth 7 and $0.8 \times 3 + 0.2 \times 13 = 5$: the program still moves left. With g , the leaves become 4, 16, 2 and 64, and the chance nodes are worth 10 and $0.8 \times 2 + 0.2 \times 64 = 14.4$: the program now moves right.

- (c) Would g change the move if the chance nodes were MIN nodes? What does this say of the evaluation functions of H-Minimax, with and without chance nodes?

Solution. No: the MIN nodes are worth $\min(2, 4) = 2$ and $\min(1, 6) = 1$, and $\min(4, 16) = 4$ and $\min(2, 64) = 2$ with g , so the program moves left in both cases. Minimax only compares values, so any order-preserving transformation of the evaluation function leaves its decisions unchanged. Expectations add values, so with chance nodes the evaluation function must be a positive linear transformation of the expected utility, such as f and unlike g .

Exercise 8 Selection in Monte Carlo tree search

A Go program chooses its moves with Monte Carlo tree search, as AlphaGo does. During a selection step, the program, player p , is to move at a node n that has been visited $N(n) = 20$ times. Its three children are candidate moves, whose wins for p and visits are given below.

Child n'	n'_1	n'_2	n'_3
Wins $Q(n', p)$	8	3	1
Visits $N(n')$	12	6	2

- (a) Which child does UCB1 select with $c = 0$? With $c = 1$? Use the natural logarithm.

Solution. The exploitation terms $Q(n', p)/N(n')$ are 0.667, 0.5 and 0.5, and the exploration terms $\sqrt{\log N(n)/N(n')}$ are 0.500, 0.707 and 1.224, with $\log 20 = 3.00$. With $c = 0$, UCB1 selects n'_1 , the best win rate. With $c = 1$, the scores are 1.166, 1.207 and 1.724, and UCB1 selects n'_3 , the least visited child.

- (b) For which values of c does UCB1 select n'_1 ?

Solution. n'_1 beats n'_3 if $0.667 + 0.500c \geq 0.5 + 1.224c$, i.e. $c \leq 0.23$, and beats n'_2 if $0.667 + 0.500c \geq 0.5 + 0.707c$, i.e. $c \leq 0.81$. UCB1 thus selects n'_1 for $c \leq 0.23$: the smaller c , the more UCB1 exploits the best win rate so far, and the larger c , the more it explores the children visited least.

QUIZ

1. In Monte Carlo Tree Search (MCTS), in the formula

$$\frac{Q(n', p)}{N(n')} + c \sqrt{\frac{\log N(n)}{N(n')}},$$

which of the following is true?

- ☒ The first term encourages the exploitation of higher-reward nodes, while the second encourages the exploration of less-visited nodes.
 - ☐ The first term encourages the exploration of less-visited nodes, while the second term encourages the exploitation of higher-reward nodes.
 - ☐ The first term encourages the exploitation of highly-visited nodes, while the second term encourages the exploration of lesser-rewarding nodes.
 - ☐ The first term encourages the exploration of lesser-rewarding nodes, while the second term encourages the exploitation of highly-visited nodes.
2. Which of the following is true in adversarial search?
- ☐ The horizon effect arises when the search is stuck in a cycle.
 - ☐ The horizon effect arises when the evaluation function is perfect.
 - ☐ The deeper in the tree the evaluation function is buried, the more the quality of the evaluation matters.
 - ☒ If not looked deep enough, bad moves may appear as good moves, because their consequences are hidden beyond the search horizon.
3. A quiescent state is
- ☐ A state in which the game will loop indefinitely.
 - ☐ A state in which an agent is stuck.
 - ☒ A state whose value is unlikely to exhibit wild swings in the near future.
 - ☐ A state in which the MAX agent is certain to win.

4. Which of the following is true of the minimax algorithm?

- ☐ It cannot fail more against a sub-optimal opponent than against an optimal one.
- ☐ It may be outperformed by a strategy that exploits a sub-optimal but predictable opponent.
- ☐ It does not ensure that MAX always wins.
- ☒ All of the above.